



Documentation Technique

Architecture · Stack technologique · Déploiement · Sécurité

Version 1.0 · ECF DWWM 2026 · Mai 2026

PHP 8.1 · MySQL 8 · MongoDB · Bootstrap 5 · XAMPP local · Heroku production

Table des matières

1. Stack technologique — Choix et justifications
2. Architecture du projet
3. Base de données — MySQL & MongoDB
4. Sécurité
5. Installation locale (XAMPP)
6. Déploiement Heroku
7. Flux de données — Pages et actions clés
8. Performances & Bonnes pratiques

1. Stack technologique

Couche	Technologie	Version	Justification
Backend	 PDO	8.1+	Langage imposé par l'ECF. PDO natif pour la sécurité (requêtes préparées) et la portabilité.
BDD relationnelle		8.0	SGBD principal. Intégrité référentielle (FK), transactions ACID pour la gestion des crédits.

Couche	Technologie	Version	Justification
BDD documentaire	MongoDB	Atlas 7	Stockage optionnel des avis (schéma flexible, requêtes analytiques). Fallback gracieux si indisponible.
Frontend CSS	Bootstrap 5.3	5.3.2	Framework CSS responsive via CDN. Grid, composants, utilitaires — compatible mobile-first.
Frontend JS	JavaScript ES6	Natif	Interactions légères (indicateur mdp, filtres auto-submit, modales). Aucun framework JS requis.
Graphiques	Chart.js	4.4	Graphiques statistiques dans l'espace admin (covoiturages/jour, crédits/jour) via CDN.
Icônes	Bootstrap Icons	1.11.3	Bibliothèque d'icônes SVG cohérente avec Bootstrap 5.
Typographie	Poppins (Google Fonts)	—	Police humaniste sans-serif, excellente lisibilité web, 4 graisses utilisées (300–700).
Versioning	Git	2.x	Git flow : main ← develop ← feature/US1 ... feature/US13. Dépôt GitHub public.
Déploiement	Heroku	—	PaaS PHP natif. JawsDB MySQL add-on. Procfile + composer.json pour le build.
Env. local	XAMPP	8.2	Apache + PHP + MySQL + phpMyAdmin sur Windows. Chemin : C:\xampp\htdocs\projet_tristan_ecoride\ecoride\

2. Architecture du projet

EcoRide adopte un pattern **MVC-lite** (sans framework) : séparation logique entre vues (pages `public/`), modèles (`src/models/`) et helpers (`src/helpers/`).

```
ecoride/ |— public/ ← Pages web accessibles | |— index.php ← Accueil (US1, US2) | |— covoiturages.php ← Recherche + filtres (US3, US4) | |— detail.php ← Fiche trajet (US5, US6) | |— inscription.php ← US7 | |— connexion.php ← Login | |— espace-utilisateur.php ← US8 (profil, rôles, véhicules, préfs) | |— saisir-voyage.php ← US9 (chauffeur) | |— historique.php ← US10 + US11 (chauffeur + passager) | |— espace-employe.php ← US12 (modération) | |— espace-admin.php ← US13 (stats, admin) | |— assets/ | |— css/style.css ← Thème custom + CSS variables | |— js/main.js ← Interactions côté client | |— img/ ← Logo, avatar-default, uploads/ | |— src/ | |— config/ | |— db_mysql.php ← getPDO() – singleton PDO + loadEnv() | | |— db_mongo.php ← getMongo() – fallback null | |— helpers/ | |— auth.php ← BASE_URL, session, rôles, CSRF | |— functions.php ← h(), flash, formatDate, starRating... | |— mail.php ← sendMail(), mailFinTrajet(), mailLitige() | |— models/ | |— UserModel.php ← CRUD utilisateur, crédits, rôles | |— CovoiturageModel.php ← Search, CRUD, stats admin | |— AvisModel.php ← Dual-write MySQL + MongoDB | |— VoitureModel.php ← Véhicules par utilisateur | |— MarqueModel.php ← getAll() pour dropdown | |— components/ | |— header.php ← <head> + Bootstrap CDN | |— navbar.php ← Navigation adaptative par rôle | |— footer.php ← Footer + Bootstrap JS | |— sql/ | |— create_tables.sql ← 10 tables + FK | |— seed_data.sql ← Données de test | |— docs/ ← Livrables ECF | |— mockups/ ← 6 maquettes HTML | |— charte-graphique.html
```

```
├─ diagrammes.html | ├─ manuel-utilisateur.html | └─ documentation-technique.html (ce fichier) | └─ .env
← Variables d'environnement (gitignore) └─ .env.example ← Template └─ .gitignore └─ Procfile ← Heroku :
web: vendor/bin/heroku-php-apache2 public/ └─ composer.json ← PHP 8.1 requirement └─ app.json ← Heroku one-
click deploy config └─ KANBAN.md └─ README.md
```

2.1 — Pattern de chargement des pages

Chaque page PHP suit le même ordre d'inclusion :

```
// 1. Session + BASE_URL + helpers
require_once '../src/helpers/auth.php';
require_once '../src/helpers/functions.php';

// 2. Modèles nécessaires (qui incluent db_mysql.php)
require_once '../src/models/CovoiturageModel.php';

// 3. Contrôle d'accès (si page protégée)
requireLogin();
requireRole('chauffeur');

// 4. Traitement POST (formulaires)
if ($_SERVER['REQUEST_METHOD'] === 'POST') { verifyCsrf(); ... }

// 5. Rendu HTML
include '../src/components/header.php';
include '../src/components/navbar.php';
// ... HTML de la page ...
include '../src/components/footer.php';
```

3. Base de données

3.1 — MySQL — Schéma des tables

Table	Colonnes clés	Relations
utilisateur	utilisateur_id, pseudo, email, mot_de_passe (bcrypt), credits, statut	1:N → voiture, covoiturage, participe
role	role_id, libelle (visiteur/passager/chauffeur/employé/administrateur)	N:N → utilisateur via utilisateur_role
utilisateur_role	utilisateur_id (FK), role_id (FK), date_attribution	Table pivot
marque	marque_id, libelle (Tesla, Renault...)	1:N → voiture
voiture	voiture_id, utilisateur_id (FK), marque_id (FK), modele, energie	N:1 → marque
covoiturage	covoiturage_id, chauffeur_id (FK), voiture_id (FK), statut, prix_par_personne	1:N → participe, avis

Table	Colonnes clés	Relations
participe	covoiturage_id (FK), passager_id (FK), statut, credits_debites	Table pivot avec statut métier
avis	avis_id, covoiturage_id (FK), auteur_id (FK), note, statut (en_attente/validé/refusé)	Dual-write vers MongoDB
parametre	parametre_id, libelle (fumeur, animaux...)	N:N → utilisateur via dispose
dispose	utilisateur_id (FK), parametre_id (FK), valeur (BOOLEAN)	Table pivot préférences
preference_libre	pref_id, utilisateur_id (FK), libelle_libre	Texte libre

3.2 — Singleton PDO

```
function getPDO(): PDO {
    static $pdo = null;
    if ($pdo === null) return $pdo;

    $pdo = new PDO(
        "mysql:host={$host};dbname={$dbname};charset=utf8mb4",
        $user, $pass,
        [
            PDO::ATTR_ERRMODE           => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES  => false, // Requêtes préparées vraies
        ]
    );
    return $pdo;
}
```

3.3 — MongoDB — Dual-write avis

À chaque insertion/modification d'avis dans MySQL, un document est synchronisé dans la collection `avis` de MongoDB Atlas :

```
// Structure document MongoDB
{
    "_id": ObjectId,
    "avis_id": 42,           // sync avec MySQL
    "covoiturage_id": 12,
    "auteur_pseudo": "marie_d",
    "note": 4,
    "commentaire": "Très bon trajet !",
    "statut": "validé",
    "date_avis": ISODate("2026-05-18")
}
```

Si la connexion MongoDB échoue, `getMongo()` retourne `null` et le code PHP vérifie avant toute écriture (pas d'erreur fatale).

4. Sécurité

OWASP A01

Contrôle d'accès

`requireLogin()` , `requireRole()` , `requireActive()` en tête de chaque page protégée. Vérification systématique que l'utilisateur connecté est propriétaire des ressources qu'il modifie (ex: `chauffeur_id = $_SESSION['user_id']`).

OWASP A02

Mots de passe

`password_hash($mdp, PASSWORD_BCRYPT)` à l'inscription.
`password_verify()` à la connexion. Politique forte imposée : ≥ 8 caractères, 1 majuscule, 1 chiffre, 1 spécial.
`session_regenerate_id(true)` après login.

OWASP A03

Injections SQL

100% PDO avec requêtes préparées (`$pdo->prepare()` + `execute([])`). `PDO::ATTR_EMULATE_PREPARES = false` force les vraies requêtes préparées côté serveur MySQL.

OWASP A03

XSS (Cross-Site Scripting)

Toute variable affichée dans le HTML passe par `h($var)` = `htmlspecialchars($str, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')` . Aucune valeur utilisateur affichée sans échappement.

OWASP A05

CSRF

Chaque formulaire POST inclut `<?= csrfField() ?>` . Le handler vérifie `hash_equals(csrfToken(), $_POST['csrf_token'])` . Token régénéré à chaque session.

OWASP A09

Variables d'environnement

Toutes les credentials (DB_PASS, MONGO_URI, MAIL_PASS) sont dans `.env` (gitignore). Le fichier `.env.example` sert de template sans valeurs sensibles.

5. Installation locale (XAMPP)

1 Cloner le projet

```
git clone https://github.com/votre-username/ecoride.git "C:/xampp/htdocs/projet_tristan_ecoride/ecoride"
```

2 Configurer les variables d'environnement

```
cp .env.example .env
# Éditer .env :
DB_HOST=localhost
DB_NAME=ecoride
DB_USER=root
DB_PASS=
```

3 Créer la base de données dans phpMyAdmin

1. Ouvrir `http://localhost/phpmyadmin`
2. Créer la base `ecoride` (charset `utf8mb4`)

3. Importer `sql/create_tables.sql` puis `sql/seed_data.sql`

4

Lancer XAMPP

Démarrer Apache et MySQL depuis le Control Panel XAMPP.

5

Accéder à l'application

```
http://localhost/projet_tristan_ecoride/ecoride/public/
```

La détection automatique de `BASE_URL` via `SCRIPT_NAME` gère le chemin avec espace.

6. Déploiement Heroku

6.1 — Prérequis

- Compte Heroku (plan Eco ou Basic)
- Heroku CLI installé : `brew install heroku/brew/heroku` (Mac) ou installateur Windows
- Add-on JawsDB MySQL (base de données cloud)

6.2 — Fichiers de déploiement inclus

Procfile

```
web: vendor/bin/heroku-php-apache2 public/
```

Indique à Heroku d'utiliser Apache comme serveur web et de servir depuis le dossier `public/`.

composer.json

```
{  
    "require": { "php": "≥8.1" }  
}
```

Spécifie la version PHP minimale. Heroku installe automatiquement la version correspondante.

6.3 — Procédure de déploiement

1 Login Heroku

```
heroku login
```

2 Créer l'application

```
heroku create ecoride-app
```

3 Ajouter JawsDB (MySQL cloud)

```
heroku addons:create jawsdb:kitefin
```

JawsDB expose automatiquement `JAWSDB_URL` comme variable d'env.

4 Configurer les variables d'environnement

```
heroku config:set DB_HOST=xxxxx.cleardb.net \  
  DB_NAME=heroku_xxxxx \  
  DB_USER=xxxxx \  
  DB_PASS=xxxxx \  
  APP_URL=https://ecoride-app.herokuapp.com \  
  MAIL_FROM=noreply@ecoride.fr
```

5 Pousser le code

```
git push heroku main
```

6 Initialiser la base de données

```
heroku run php -r "..."  
# Ou importer via MySQL Workbench connecté à JawsDB
```

Note sessions : Heroku peut réinitialiser le système de fichiers à chaque déploiement. Les uploads de photos utilisateurs sont perdus. En production, utiliser un service comme AWS S3 ou Cloudinary pour le stockage des fichiers.

7. Flux de données — Actions clés

7.1 — Recherche de covoiturages

```
GET /covoiturages.php?depart=Paris&arrivee=Lyon&date=2026-05-20  
  &eco=1&prix_max=20&duree_max=4&e_min=4  
  
→ CovoiturageModel::search($filters)  
→ SELECT cov.*, u.pseudo, u.photo, AVG(a.note) as note_moy  
  FROM covoiturage cov  
  JOIN utilisateur u ON cov.chauffeur_id = u.utilisateur_id  
  LEFT JOIN avis a ON a.covoiturage_id = cov.covoiturage_id AND a.statut = 'valide'  
  JOIN voiture v ON cov.voiture_id = v.voiture_id  
  WHERE cov.lieu_depart LIKE :dep  
    AND cov.lieu_arrivee LIKE :arr  
    AND cov.date_depart = :date  
    AND cov.statut = 'planifié'  
    AND cov.nb_places > 0  
    [AND v.energie = 'electrique'] ← si filtre eco  
    [AND cov.prix_par_personne ≤ :prix_max]  
    [AND TIMESTAMPDIFF(MINUTE, cov.heure_depart, cov.heure_arrivee) ≤ :duree_max]
```



```
HAVING [note_moy ≥ :note_min]
ORDER BY cov.date_depart, cov.heure_depart
```

7.2 — Transaction de réservation

```
try {
    getPDO()→beginTransaction();

    // 1. Insérer la participation
    $stmt = getPDO()→prepare(
        "INSERT INTO participe (covoiturage_id, passager_id, statut, credits_debites)
        VALUES (:cov, :pass, 'reserve', :credits)"
    );

    // 2. Déduire les crédits du passager
    getPDO()→prepare(
        "UPDATE utilisateur SET credits = credits - :prix WHERE utilisateur_id = :id"
    )→execute([ ... ]);

    // 3. Décrémenter les places
    getPDO()→prepare(
        "UPDATE covoiturage SET nb_places = nb_places - 1 WHERE covoiturage_id = :id"
    )→execute([ ... ]);

    getPDO()→commit();
} catch (Exception $e) {
    getPDO()→rollback();
    setFlash('error', 'Erreur lors de la réservation.');
```

8. Performances & Bonnes pratiques

Pratique	Implémentation
Connexion BDD singleton	<code>static \$pdo</code> dans <code>getPDO()</code> — une seule connexion par requête HTTP
CDN pour les assets lourds	Bootstrap, Bootstrap Icons, Poppins, Chart.js — tous via CDN (cache navigateur)
Requêtes préparées réutilisées	Les modèles préparent une fois et exécutent N fois
Pas de N+1 query	Les jointures SQL récupèrent le conducteur et la note en une seule requête
Variables CSS	<code>--eco-dark</code> , <code>--eco-main</code> ... — palette centralisée, changeable en une ligne
BASE_URL dynamique	Détection via <code>SCRIPT_NAME</code> regex — fonctionne dans tous les environnements (local, Heroku)
Flash messages	Un seul message en session, lu une fois puis supprimé — pas de duplication
MongoDB fallback	<code>getMongo()</code> retourne <code>null</code> si l'extension PHP MongoDB est absente — pas d'erreur fatale

EcoRide · Documentation Technique v1.0 · ECF DWWM 2026

Pour exporter en PDF : Ctr1+P → *Enregistrer en PDF*